

```
ML 1 Uber.csv,  
from google.colab import drive  
drive.mount('/content/drive')
```

```
!pip install kagglehub
```

```
import kagglehub
```

```
path = kagglehub.dataset_download("yasserh/uber-fares-dataset")  
print("Path to dataset files:", path)
```

```
import os  
import pandas as pd  
path = "/root/.cache/kagglehub/datasets/yasserh/uber-fares-dataset/versions/1"  
print(os.listdir(path))
```

```
uber = pd.read_csv(path + "/uber.csv")
uber.head()
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
uber.to_csv("/content/drive/MyDrive/DataFiles/uber.csv", index=False)
print("✅ Saved uber.csv in your Google Drive → DataFiles folder")
```

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
# ✅ Step 1: Remove unnecessary columns (if they exist)
if 'Unnamed: 0' in uber.columns:
    uber_2 = uber.drop(['Unnamed: 0'], axis=1)
else:
    uber_2 = uber.copy()
```

✅ Step 2: Check and drop missing values

```
print("Missing values before cleaning:\n", uber_2.isnull().sum())

uber_2.dropna(inplace=True)

print("\nMissing values after cleaning:\n", uber_2.isnull().sum())
```

✅ Step 3: Define a Haversine function (to calculate distance between two points)

```
def haversine(lon1, lon2, lat1, lat2):
    """
    Calculate the great circle distance between two points on Earth (in kilometers)
    """
    lon1, lon2, lat1, lat2 = map(np.radians, [lon1, lon2, lat1, lat2])
    diff_lon = lon2 - lon1
    diff_lat = lat2 - lat1
    km = 2 * 6371 * np.arcsin(
        np.sqrt(np.sin(diff_lat/2.0)**2 +
                np.cos(lat1) * np.cos(lat2) * np.sin(diff_lon/2.0)**2)
    )
    return km
```

✅ Step 4: Calculate the distance for each trip

```
uber_2['Distance'] = haversine(
    uber_2['pickup_longitude'], uber_2['dropoff_longitude'],
    uber_2['pickup_latitude'], uber_2['dropoff_latitude']
)
```

✅ Step 5: Round off and convert to float

```
uber_2['Distance'] = uber_2['Distance'].astype(float).round(2)
```

✅ Step 6: Preview the dataset

```
uber_2.head()
```

```
# Plot: Fare vs Distance

plt.figure(figsize=(7,5))

plt.scatter(uber_2['Distance'], uber_2['fare_amount'], alpha=0.5)

plt.xlabel('Distance (km)')

plt.ylabel('Fare Amount ($)')

plt.title('Uber Fare vs Distance')

plt.show()
```

```
# ✅ Data Cleaning (remove outliers and invalid data)

# Remove trips with unrealistic distances or fares

uber_2.drop(uber_2[uber_2['Distance'] > 60].index, inplace=True)

uber_2.drop(uber_2[uber_2['Distance'] == 0].index, inplace=True)

uber_2.drop(uber_2[uber_2['fare_amount'] <= 0].index, inplace=True)

uber_2.drop(uber_2[(uber_2['fare_amount'] > 100) & (uber_2['Distance'] < 1)].index,
inplace=True)

uber_2.drop(uber_2[(uber_2['fare_amount'] < 100) & (uber_2['Distance'] > 100)].index,
inplace=True)

# Check dataset info

uber_2.info()
```

```
# ✅ Convert pickup_datetime column to datetime format
uber_2['pickup_datetime'] = pd.to_datetime(uber_2['pickup_datetime'])

# Extract useful date & time features
uber_2['Year'] = uber_2['pickup_datetime'].dt.year
uber_2['Month'] = uber_2['pickup_datetime'].dt.month
uber_2['Day'] = uber_2['pickup_datetime'].dt.day
uber_2['Hour'] = uber_2['pickup_datetime'].dt.hour
uber_2['DayOfWeek'] = uber_2['pickup_datetime'].dt.dayofweek # Monday = 0

# Map numeric days to names
day_map = {0:'Mon', 1:'Tue', 2:'Wed', 3:'Thu', 4:'Fri', 5:'Sat', 6:'Sun'}
uber_2['DayName'] = uber_2['DayOfWeek'].map(day_map)

uber_2.head()
```

```
# ✅ Visualization Section

# Number of trips per Year

yearly_trips = uber_2.groupby('Year').size()

plt.bar(yearly_trips.index, yearly_trips.values, color='teal')

plt.xlabel('Year')

plt.ylabel('Number of Trips')
```

```
plt.title('Average Uber Trips per Year')  
plt.show()
```

```
# Number of trips per Month  
monthly_trips = uber_2.groupby('Month').size()  
plt.bar(monthly_trips.index, monthly_trips.values, color='orange')  
plt.xlabel('Month')  
plt.ylabel('Number of Trips')  
plt.title('Average Uber Trips per Month')  
plt.show()
```

```
# Number of trips per Day of the Week  
day_trips = uber_2.groupby('DayName').size()  
plt.bar(day_trips.index, day_trips.values, color='purple')  
plt.xlabel('Day of Week')  
plt.ylabel('Number of Trips')  
plt.title('Average Uber Trips per Day')  
plt.show()
```

```
#  Linear Regression Model
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Independent (X) and Dependent (y) variables
```

```
X = uber_2['Distance'].values.reshape(-1, 1)
```

```
y = uber_2['fare_amount'].values.reshape(-1, 1)
```

```
# Standardization
```

```
scaler_X = StandardScaler()
```

```
scaler_y = StandardScaler()
```

```
X_scaled = scaler_X.fit_transform(X)
```

```
y_scaled = scaler_y.fit_transform(y)
```

```
# Split the dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_scaled, test_size=0.2,  
random_state=0)
```

```
# Train the model
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
# Model Scores
```

```
print(f"Training Score: {model.score(X_train, y_train):.3f}")
```

```
print(f"Testing Score: {model.score(X_test, y_test):.3f}")
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

```
# Actual vs Predicted plot
```

```
plt.figure(figsize=(6,4))
```

```
plt.scatter(y_test, y_pred, alpha=0.3, color='red')
```

```
plt.xlabel("Actual (Standardized)")
```

```
plt.ylabel("Predicted (Standardized)")
```

```
plt.title("Linear Regression: Fare vs Distance")
```

```
plt.show()
```

```
#  Statistics Summary
```

```
import statistics as st
```

```
print("Mean Fare:", round(st.mean(uber_2['fare_amount']), 2))
```

```
print("Median Fare:", round(st.median(uber_2['fare_amount']), 2))
```

```
print("Std Dev Fare:", round(st.stdev(uber_2['fare_amount']), 2))
```

```
print("\nMean Distance:", round(st.mean(uber_2['Distance']), 2))
```

```
print("Median Distance:", round(st.median(uber_2['Distance']), 2))
```

```
print("Std Dev Distance:", round(st.stdev(uber_2['Distance']), 2))
```

```
#  Mount Google Drive (if dataset is stored there)
```

```
from google.colab import drive
```



```
drive.mount('/content/drive')
```